

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: НОВЫЕ ЯЗЫКИ ПРОГРАММИРОВАНИЯ

Студент гр. 4345

М.В. Наумов

Студентка гр. 4343

Е.П. Попова

Студентка гр. 4343

А.Р. Резяпова

Руководитель

Р.П. Шестопапов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: М.В. Наумов Группа: 4345

Студент: Е.П. Попова Группа: 4343

Студент: А.Р. Резяпова Группа: 4343

Тема практики: Разработка визуализатора алгоритма Прима на языке Java

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Java с графическим интерфейсом.

Алгоритм: алгоритм Прима.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	М.В. Наумов
---------	-------------

Студентка	Е.П. Попова
-----------	-------------

Студентка	А.Р. Резяпова
-----------	---------------

Руководитель	Р.П. Шестопалов
--------------	-----------------

АННОТАЦИЯ

Данный отчёт описывает процесс разработки. Программа по выполнению и визуализации алгоритма Прима была разработана при помощи языка Java и библиотеки Swing, были добавлены широкие возможности для редактирования графа, а также для его сохранения и загрузки, вся работа была распределена по ролям, что позволило добиться всех поставленных целей.

SUMMARY

This report describes the development process. The program for executing and visualizing Prim's algorithm was developed using the Java language and the Swing library; extensive capabilities for editing the graph, as well as for saving and loading it, were added; all work was distributed among roles, which made it possible to achieve all the set goals.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Вводное задание:	7
2.2. Составление спецификации программы и плана разработки.....	8
2.2.1 Исходные требования к программе	8
2.2.1.1 Требования к вводу исходных данных.....	8
2.2.1.2 Требования к визуализации.....	8
2.2.1.3 Требования к управлению алгоритмом.....	9
2.2.1.4 Требования к обработке ошибок.....	9
2.2.2 План разработки и распределение ролей в бригаде.....	9
2.2.3 Уточнение требований после сдачи прототипа.....	10
2.2.4 Уточнение требований после сдачи 1-й версии	11
2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом	11
2.3.1. Структуры данных.....	11
2.3.2. Основные методы	12
2.3.3. Тестирование.....	13
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	18
4 ОТЗЫВ РУКОВОДИТЕЛЯ	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
ПРИЛОЖЕНИЕ А	23

ВВЕДЕНИЕ

Предметная область:

Предметной областью при написании ядра программы является теория графов, одну из задач которой программа и решает. Программа оптимально строит минимальное остовное дерево, что может пригодиться в электронике или при составление оптимальных маршрутов.

Цель практики: Целью учебной практики является формирование практических навыков командной разработки программного обеспечения на примере создания интерактивного визуализатора алгоритмов на графах. В рамках работы над проектом осуществлялось разделение ответственности между участниками команды: разработка модуля ядра и алгоритмической части, создание модуля управления и интеграции компонентов, а также проектирование и реализация пользовательского интерфейса с системой пошаговой визуализации выполнения алгоритма.

Задачи практики:

В ходе практики были поставлены и решены следующие задачи:

- изучение языка программирования Java;
- реализация модуля ядра алгоритма, включающего структуры данных для представления графа (вершины, рёбра) и алгоритм Прима для построения минимального остовного дерева;
- разработка графического интерфейса для визуализации графа и пошагового отображения работы алгоритма;
- реализация модуля управления и интеграции, обеспечивающего связь между ядром алгоритма и интерфейсом, обработку событий пользователя, управление пошаговым выполнением алгоритма с возможностью отката на предыдущий шаг, загрузку и сохранение данных;
- обеспечение обработки исключительных ситуаций и некорректного ввода пользователя;
- командная разработка с использованием системы контроля версий Git и итеративный подход к созданию программного продукта.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Визуализация алгоритмов широко применяется в образовании для наглядного изучения сложных алгоритмических концепций. Алгоритм Прима используется при решении задач построения минимальных остовных деревьев в области проектирования сетей, логистики, прокладке дорог и трубопроводов и т.д.

1.2 Задачи практики

- изучение языка программирования Java;
- реализация алгоритма Прима и структур данных для работы с графами;
- разработка графического интерфейса;
- создание модуля управления (Controller) для связи интерфейса с алгоритмом;
- реализация пошагового выполнения алгоритма с возможностью отката на предыдущий шаг;
- обеспечение загрузки графа из файла и сохранения результата;
- обработка исключительных ситуаций и некорректного ввода пользователя;
- взаимодействие в команде через Git и поэтапная разработка программного продукта.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание:

Наумов Матвей Валерьевич:

В качестве вводного задания была переделана программа по выполнению алгоритма Ахо-Корасика на языке Java, для этой программы была реализована структура данных: бор. Программа принимает на вход исходную строку, искомую строку, символ-джокер, символ, в который “джокер” не может превращаться. Пример работы программы представлен в таблице 1.

Таблица 1 – Пример работы программы

№ п/п	Входные данные	Выходные данные	Комментарии
	ACTANCA A\$\$A \$ G	1 4	Простой пример, решения находятся корректно
	ACTANCA A\$\$A\$ \$ T		Вариант с запретом отсеивается
	ACTANCA A\$\$A \$ T	4	Варианты с запретом успешно отсеиваются, а иные варианты разрешаются

В рамках вводного задания студентка Попова Елизавета прошла онлайн-курс по основам Java, в ходе которого были изучены: базовый синтаксис языка и примитивные типы данных; объектно-ориентированное программирование (наследование, абстрактные классы, пакеты); обработка исключений и логирование ошибок; работа с файловой системой и чтение/запись файлов.

В рамках работы студенткой Резяповой Алиной была переписана лабораторная работа №5 «Алгоритм Ахо-Корасик» с C++ на Java. Программа ищет все вхождения набора образцов в тексте за один проход. Вход: текст, число образцов и сами образцы. Выход: позиции и номера вхождений, количество вершин в автомате. Структуры данных реализованы вручную, библиотечные контейнеры не использовались.

Входной файл input.txt:

ACGTAACGT

3

ACG

GTA

CGT

Результат:

Результаты поиска

1 1

2 3

3 2

6 1

7 3

Количество вершин: 10

2.2. Составление спецификации программы и плана разработки

2.2.1 Исходные требования к программе

2.2.1.1 Требования к вводу исходных данных

Программа поддерживает два способа ввода исходных данных: загрузку из файла в формате списка смежности и интерактивный ввод — создание и редактирование графа непосредственно в интерфейсе с использованием мыши (добавление вершин кликом по холсту, соединение вершин рёбрами, удаление элементов). Отсутствие входного файла допускается, что позволяет пользователю создать нужный граф с нуля.

2.2.1.2 Требования к визуализации

В соответствии с требованиями практики, программа должна обеспечивать наглядное представление работы алгоритма:

- отображение графа с подписями вершин и весов рёбер;
- цветовое выделение элементов на каждом шаге: текущее ребро, посещённые вершины и рёбра, вошедшие в минимальное остовное дерево;
- отображение веса построенного дерева после завершения алгоритма.

2.2.1.3 Требования к управлению алгоритмом

Программа должна поддерживать пошаговое выполнение алгоритма Прима с возможностью перехода как вперёд, так и назад по шагам. Реализована блокировка кнопок в неактивных состояниях (например, невозможность перехода назад на первом шаге). Предусмотрена возможность сохранения результата работы алгоритма в текстовый файл.

2.2.1.4 Требования к обработке ошибок

Приложение должно быть устойчивым к некорректным действиям пользователя: загрузка неверного формата файла, пустого файла, несвязного графа, попытка выполнения действий без загруженного графа. Все ошибки выводятся в виде понятных сообщений, программа не завершается аварийно.

2.2.2 План разработки и распределение ролей в бригаде

Разработка велась в соответствии с планом, представленном в таблице 2.

Таблица 2 - План разработки

Версия	Срок	Содержание
Прототип	10.07	Графическое представление интерфейса без привязки к логике
Альфа-версия	13.07	Загрузка графа из файла, отображение, запуск алгоритма с выводом результата

Бета-версия	15.07	Пошаговое выполнение алгоритма, визуализация шагов, сохранение результата
Финальная версия	17.07	Блокировка кнопок, кнопка "О разработчиках", полная обработка ошибок

Распределение ролей в бригаде:

Наумов Матвей — модуль ядра и алгоритма (Model). Ответственность: реализация структур данных (граф, вершины, рёбра, бинарная куча), алгоритма Прима с историей шагов, парсера файлов и сохранения результата.

Попова Елизавета — пользовательский интерфейс и визуализация (View). Ответственность: разработка графического интерфейса (главное окно, панели управления, область отрисовки), отрисовка графа и визуализация шагов алгоритма с цветовыми выделениями.

Резяпова Алина — управление и интеграция (Controller). Ответственность: создание связующего звена между ядром и интерфейсом, обработка событий от кнопок и меню, управление пошаговым выполнением алгоритма, блокировка кнопок в зависимости от состояния, обработка ошибок.

2.2.3 Уточнение требований после сдачи прототипа

После сдачи прототипа было принято решение добавить возможность редактирования графа непосредственно через графический интерфейс (добавление/удаление вершин и рёбер по клику мыши), что позволило пользователю не только загружать граф из файла, но и создавать его с нуля. Также было решено реализовать панель истории для отображения списка

шагов алгоритма и стало понятно, что сохраняться должен сам результат, а не граф, что было исправлено в следующей версии.

2.2.4 Уточнение требований после сдачи 1-й версии

На этом этапе было добавлено требование приведения названий кнопок к одному языку, последовательного выполнения алгоритма, размещение окон для ввода данных по центру экрана. Были указаны некоторые требования к обработке исключений, которые впоследствии были полностью выполнены в финальной версии.

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

2.3.1. Структуры данных

Core:

Vertex – реализация вершины, хранит рёбра и свой номер.

Edge – реализация ребра, связывает вершины, храня их, а также хранит свой вес

Graph – реализация графа, хранит рёбра и вершины, именно над ним и выполняются манипуляции

PrimAlgorithm – даёт возможность пошагово проходить алгоритм Прима

MinHeap – реализация двоичной мин кучи, применяется в алгоритме Прима, помогает получать ребро с меньшим весом

Runner – считывает граф и подготавливает всё для запуска

View:

MainWindow – главное окно приложения, контейнер для всех UI-компонентов.

VertexData – хранит идентификатор вершины (id) и её координаты на холсте.

EdgeData – хранит идентификаторы начальной (from) и конечной (to) вершин и вес ребра (weight).

В классе GraphCanvas для отслеживания визуального состояния алгоритма используется Set<Integer> (для посещённых вершин) и Set<String>

(для рёбер минимального остовного дерева), что позволяет быстро проверять принадлежность элемента к множеству.

В классе `ToolbarPanel` реализованы кнопки инструментов: добавления и стирание вершин и рёбер, очистка всего холста.

`BottomPanel` – Класс, хранящий кнопки по управлению анимации графа, историю и кнопку о разработчиках.

`ToastNotification` – класс, отвечающий за всплывающие уведомления.

2.3.2. Основные методы

Основные методы пакета Core:

В ядре основными являются методы класса `Graph`: `addVertex`, `removeVertex`, `addEdge`, `removeEdge`. Так как именно они предоставляют доступ к редактированию графа. Также основными методами можно назвать методы класса `PrimAlgorithm`: `start`, `prevStep` и `nextStep`. Ведь благодаря ним осуществляется движение по алгоритму.

Основные методы пакета View:

В модуле пользовательского интерфейса ключевыми являются методы класса `GraphCanvas`: `initMouseHandlers()` для обработки событий мыши (клики, панорамирование, зум), `getVertexAtPoint()` и `getEdgeAtPoint()` для определения элемента под курсором, `paintComponent()` с методами `drawEdges()` и `drawVertices()` для кастомной отрисовки графа через `Graphics2D`, а также `updateState()` для визуализации состояния алгоритма. Класс `ToolbarPanel` содержит методы управления режимами (`setAddVertexMode()`, `setEraserMode()`) и диалоговые методы (`showAddVertexDialog()`, `showWeightDialog()`). Класс `ToastNotification` реализует метод `show()` для отображения кратких уведомлений в правом нижнем углу с автоматическим исчезновением. DTO-классы `VertexData` и `EdgeData` обеспечивают передачу данных между ядром и интерфейсом.

Основные методы класса Controller:

`loadGraphFromFile()` — загрузка графа из файла через диалоговое окно;
`onStart()` — инициализация и запуск алгоритма Прима;

onNextStep() — выполнение одного шага алгоритма вперёд;
onPrevStep() — возврат на один шаг назад;
saveResultToFile() — сохранение результата работы алгоритма в текстовый файл;
onReset() — сброс приложения в исходное состояние;
showAuthors() — отображение информации о разработчиках.

2.3.3. Тестирование

2.3.3.1. Тестирование графического интерфейса

Запуск программы представлен на рисунке 1. (см. рис. 1)

Загрузка графа из файла и выполнение алгоритма представлены на рисунках ниже. (см. рис. 2, рис. 3, рис. 4, рис. 5, рис. 6, рис. 7).

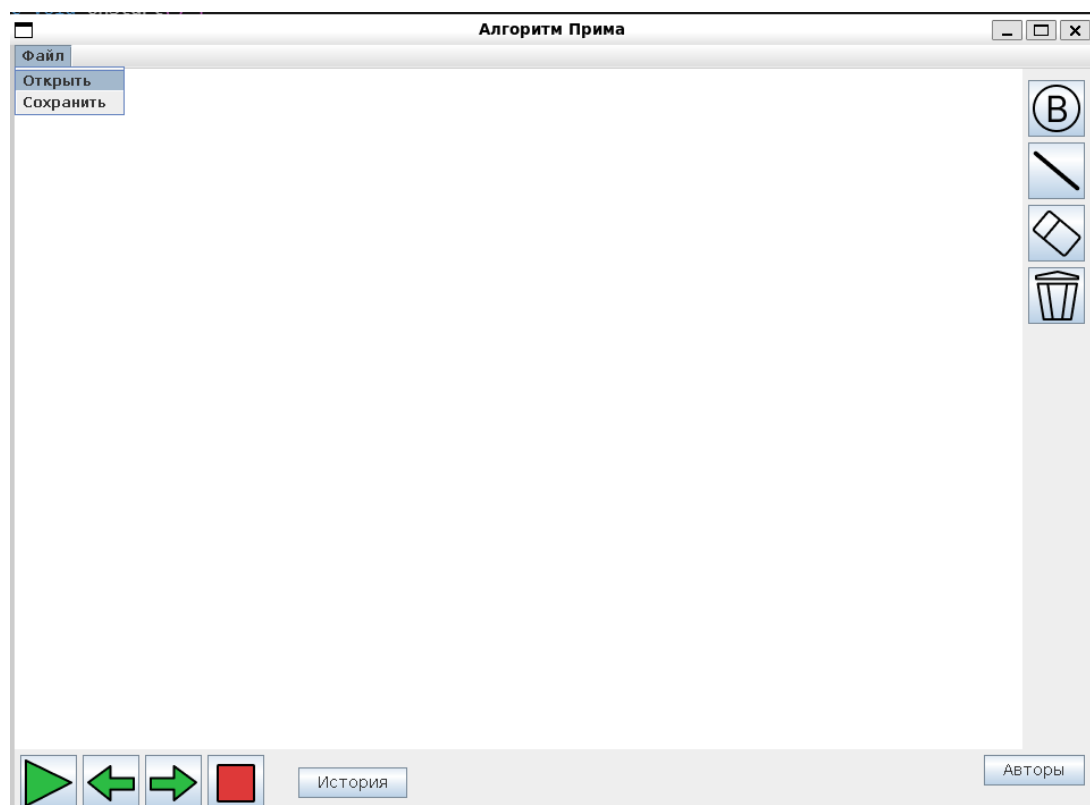


Рисунок 1 - Запуск программы

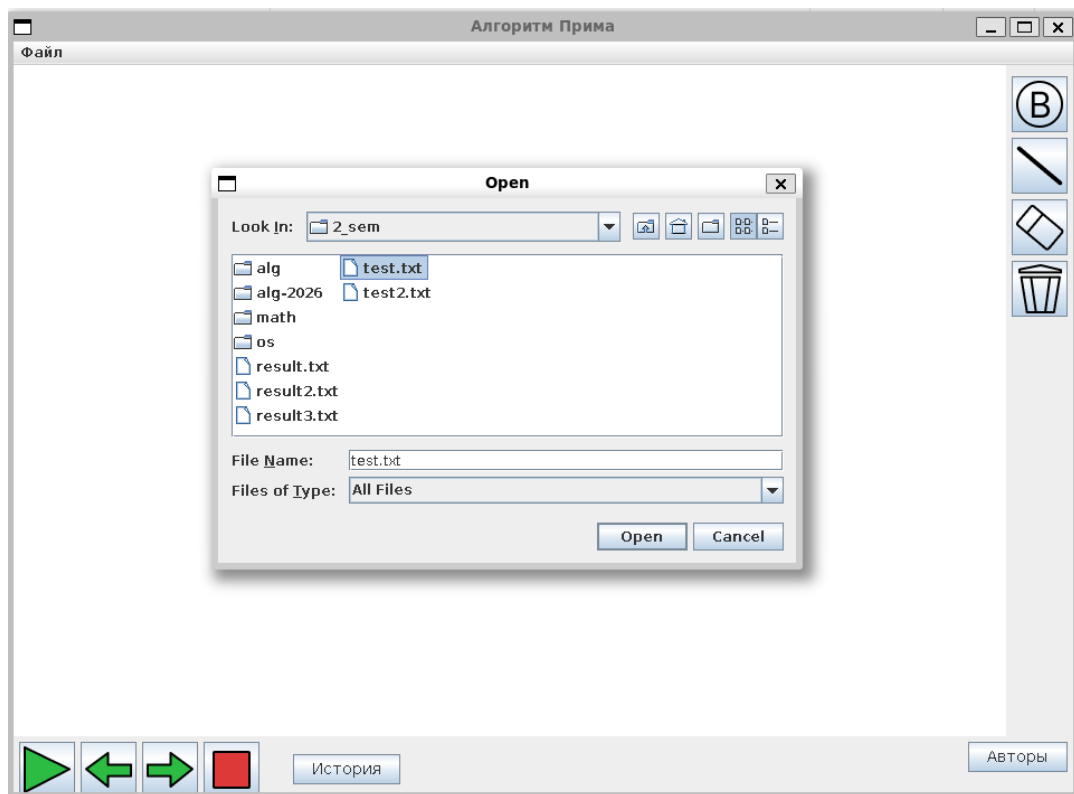


Рисунок 2 - Загрузка графа из файла

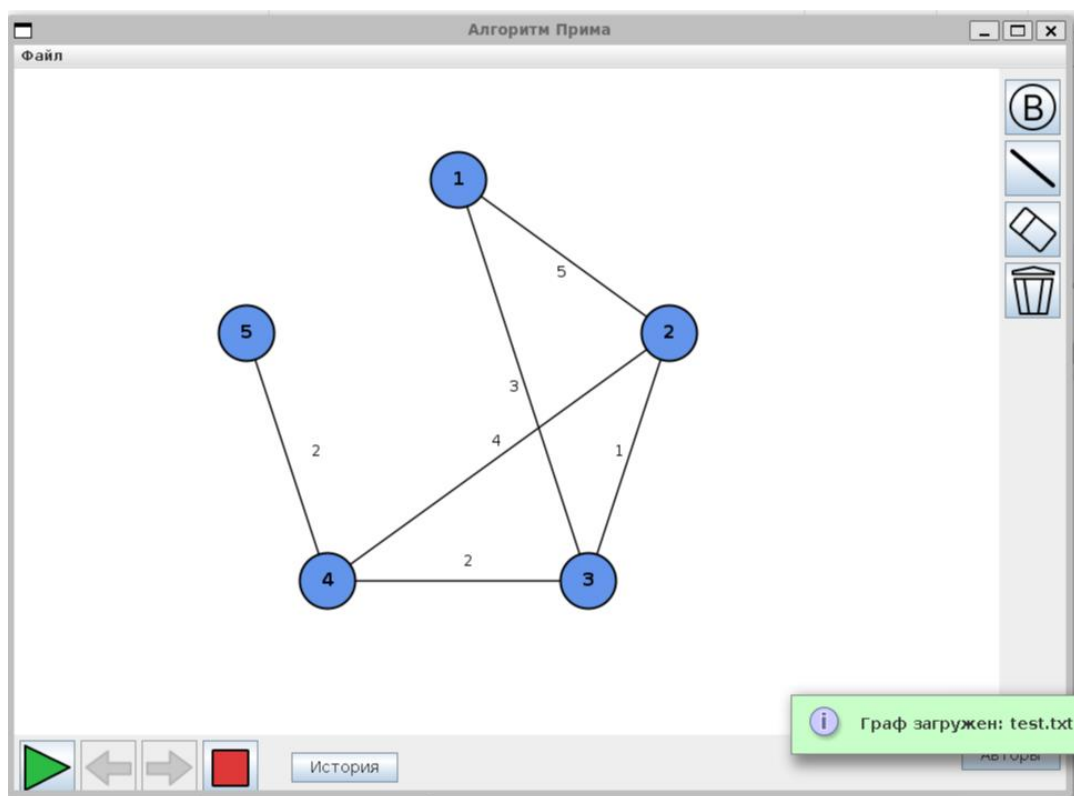


Рисунок 3 - исходный граф загружен

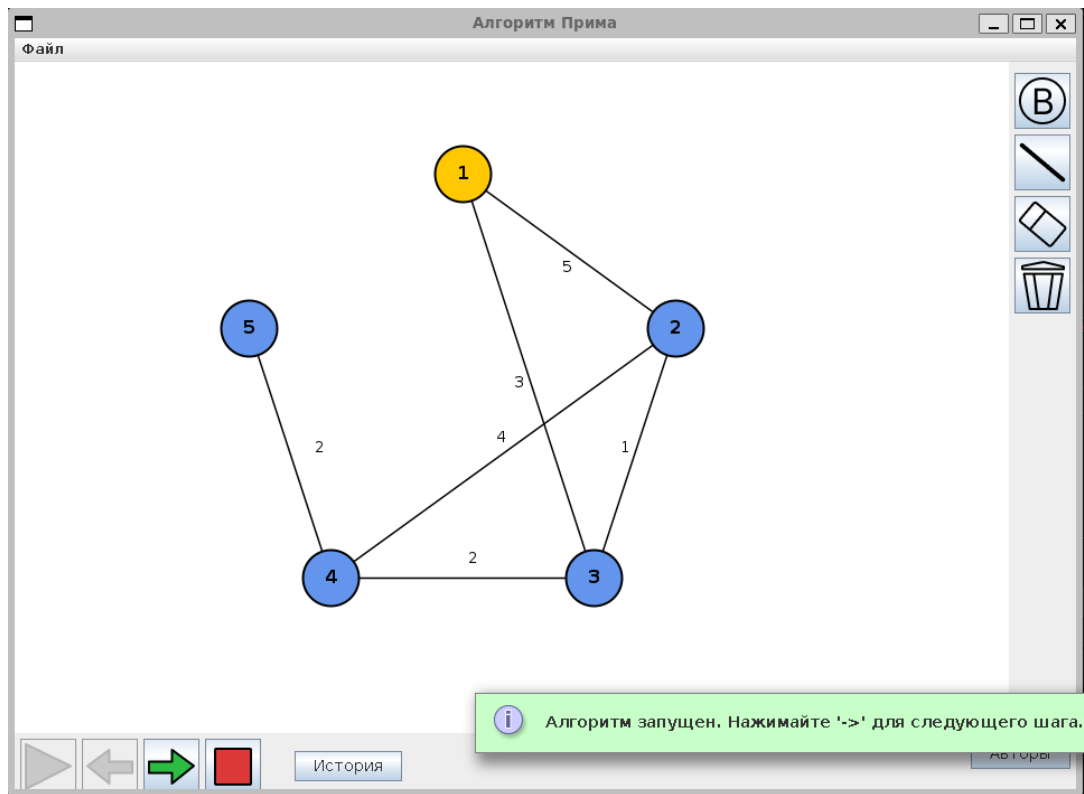


Рисунок 4 - Начало работы алгоритма

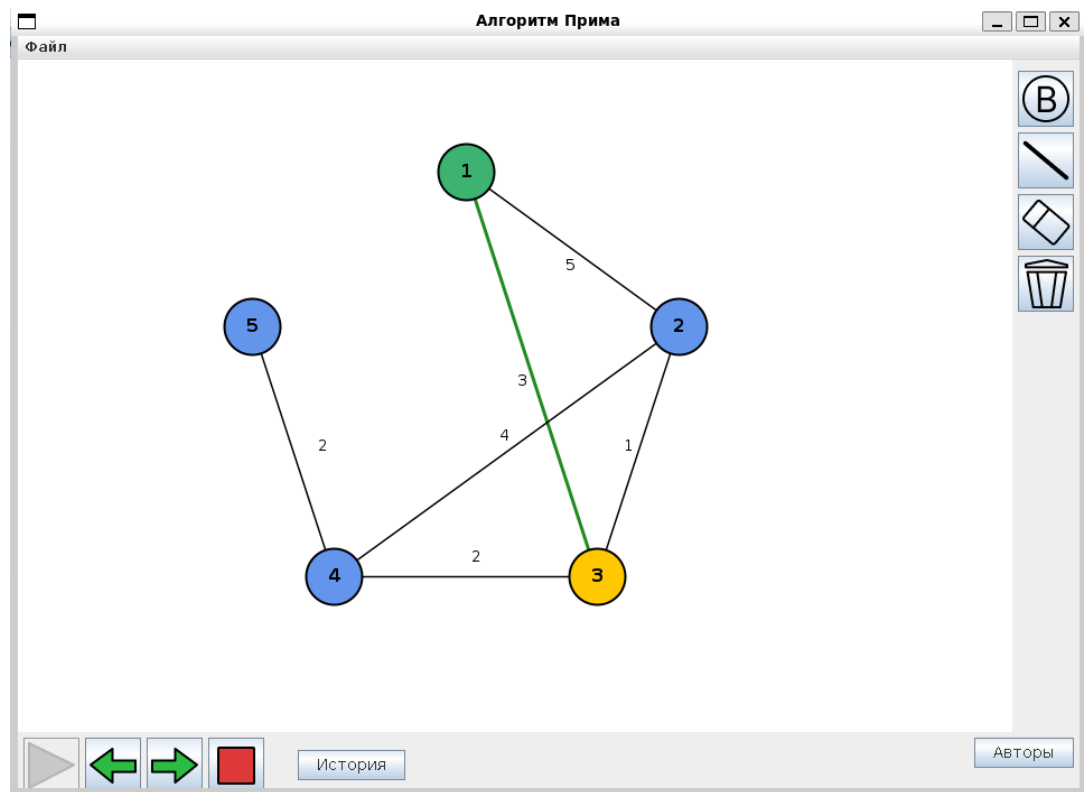


Рисунок 5 - следующий шаг работы алгоритма

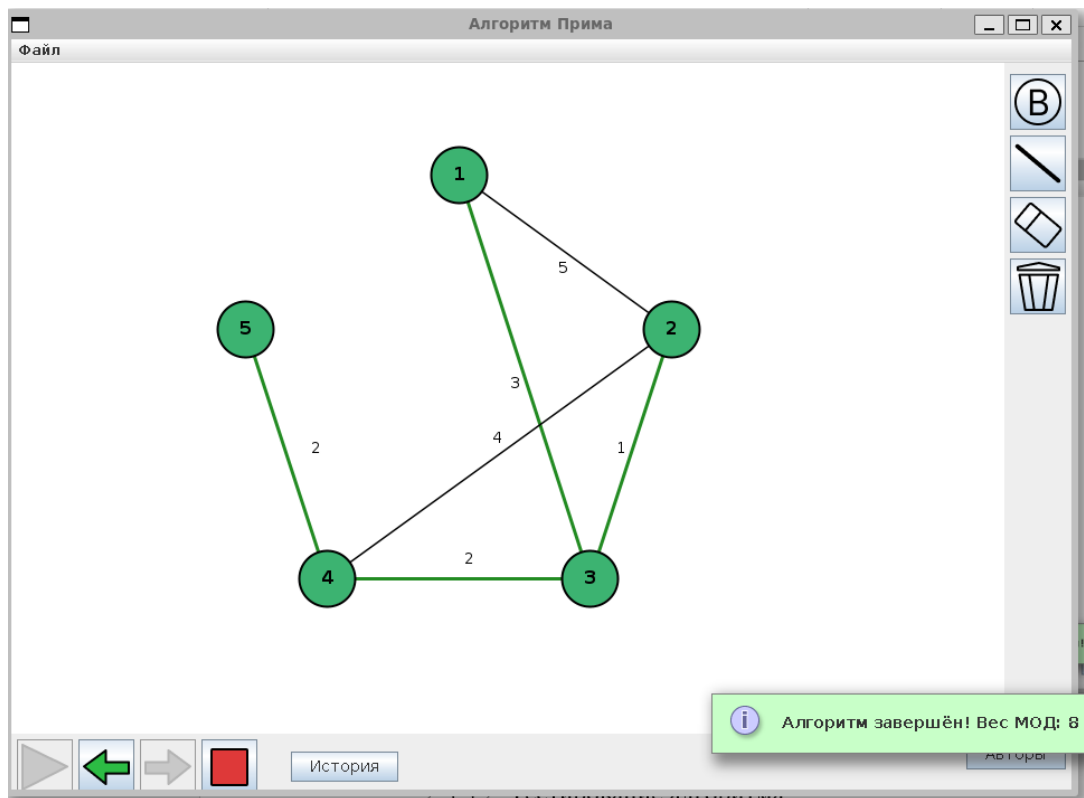


Рисунок 6 - конец работы алгоритма

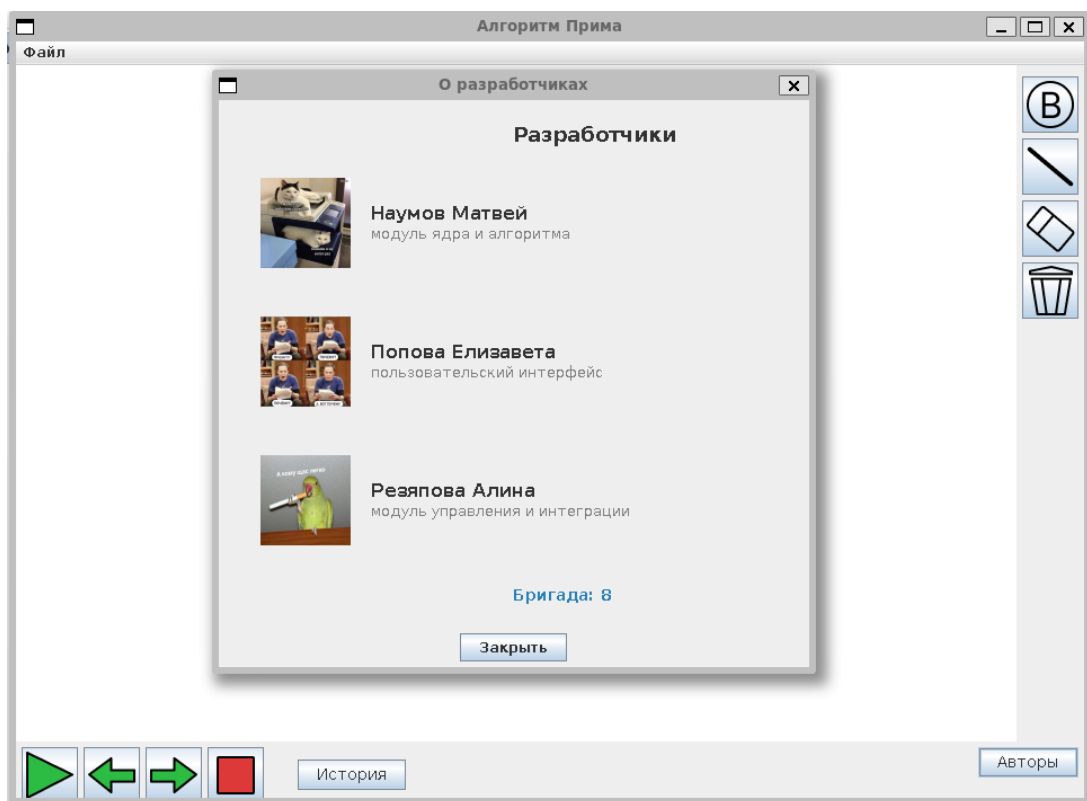


Рисунок 7 - Кнопка об авторах

2.3.3.2. Тестирование алгоритма

Тестирование алгоритмической части проводилось в основном перед альфой, поскольку последующие изменения практически не влияли на базовую логику вычислений. В основном тестирование происходило вручную путём промежуточных выводов, которые впоследствии были стёрты, и различных файлов.

Проверка корректности работы охватывала основные проблемы, алгоритм был проверен на дереве, на графе из одной вершины, на дереве, на обычном графе, который показывает корректные выборы, а также на несвязном графе. А также перед тестами различными способами добавлялись и удалялись вершины. Во всех тестах алгоритм отработал корректно, полностью соответствуя ожидаемому результату.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Язык программирования: Java — основной язык реализации проекта. Выбор обусловлен кроссплатформенностью, наличием встроенных средств для создания графического интерфейса и широким распространением в образовательной среде.

Графическая библиотека: Swing — использовалась для разработки графического интерфейса приложения. Swing предоставляет необходимые компоненты для создания окон, панелей, кнопок, области отрисовки и обработки событий мыши и клавиатуры.

Среды разработки: VS Code, IntelliJ IDEA Community Edition — использовались для написания кода, отладки и запуска приложения.

Система контроля версий и хостинг репозитория: Git и GitHub — использовались для командной разработки, отслеживания изменений в коде, синхронизации работы между участниками бригады и хранения проекта в удалённом репозитории.

Инструменты документирования: Microsoft Word — применялся для оформления отчётной документации в соответствии с требованиями кафедры.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на Отлично.
Отзыв руководителя с оценкой (см. рис. 8).

Отзыв
о прохождении учебной практики

Наумов Матвей Валерьевич, студент группы 4345, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил учебную практику по теме “Новые языки программирования” с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Наумов Матвей Валерьевич выполнял задание по написанию ядра программы, реализующей пошаговую работу алгоритма Прима с широкими возможностями по редактированию графа.

В результате практики обучающийся успешно написал ядро программы, реализовав классы, представляющие основные понятия, структуры и сам алгоритм, справившись с поставленным заданием. Получаемую работу обучающийся Наумов Матвей Валерьевич выполнял качественно и точно в сроки.

По результатам работы Наумова Матвея Валерьевича учебную практику оцениваю на Отлично.

Руководитель практики
Ассистент каф. МОЭВМ 15.07.2026  /Шестопалов Р.П.

Рисунок 8 - Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была разработана программа для визуализации работы алгоритма Прима на языке Java с графическим интерфейсом.

Решены все поставленные задачи: изучен язык Java [1], реализованы структуры данных и алгоритм Прима, разработан графический интерфейс на Swing [2], создан модуль управления (Controller) для связи интерфейса с алгоритмом, обеспечена пошаговая визуализация с возможностью отката, а также обработка ошибок. В процессе работы были изучены способы добавления изображений в Swing-приложение [3], а также использованы инструменты для работы с иконками интерфейса [4]. Для проектирования отдельных элементов интерфейса был проанализирован опыт работы со Scene Builder [5]. При разработке интерфейса учитывались особенности выбора фреймворка в Java [6]. Дополнительно были изучены практические руководства по работе с графикой в Java [7] и гайды по созданию иконок для интерфейса [8]. Исходный код разработанного приложения размещён в открытом репозитории на GitHub [9].

Приложение позволяет загружать и редактировать граф, запускать алгоритм, проходить его по шагам и сохранять результат. Цель практики достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/187/syllabus> (дата обращения: 06.07.2026).
2. Swing vs AWT: разница и выбор фреймворка в Java [Электронный ресурс]. URL: <https://sky.pro/wiki/java/swing-vs-awt-raznitsa-i-vybor-freymvorka-v-java/> (дата обращения: 06.07.2026).
3. How to put an IMAGE on to JPanel/JFrame in Java Swing [Электронный ресурс]. URL: <https://www.youtube.com/watch?v=dFOIoX3fXpQ> (дата обращения: 17.07.2026).
4. Гайд по работе с иконками в Figma и их передачей [Электронный ресурс]. URL: <https://uxrdsn.ru/project-icons?ysclid=mrnnovrp90579177921> (дата обращения: 17.07.2026).
5. JavaFX Scene Builder Information [Электронный ресурс]. URL: <https://www.oracle.com/java/technologies/javase/javafxscenebuilder-info.html> (дата обращения: 08.07.2026).
6. Работа с графическим интерфейсом в Джава: списки и Swing OTUS [Электронный ресурс]. URL: <https://otus.ru/journal/rabota-s-graficheskim-interfejsom-v-dzhava-spiski-i-swing/?ysclid=mrozqu3nd5521988592> (дата обращения: 07.07.2026).
7. Add An Image To JPanel In Swing – Javaexercise [Электронный ресурс]. URL: <https://www.javaexercise.com/java/add-an-image-to-a-jpanel> (дата обращения: 16.06.2026).
8. How to set icon image for swing application? [Электронный ресурс]. URL: <https://stackoverflow.com/questions/7194734/how-to-set-icon-image-for-swing-application> (дата обращения: 16.06.2026).
9. Репозиторий проекта на GitHub [Электронный ресурс]. URL: https://github.com/ваш_логин/название_репозитория (дата обращения: 17.07.2026).

ПРИЛОЖЕНИЕ А

На рисунке 11 представлен итоговый вид графического интерфейса приложения. (см. рис. 11)

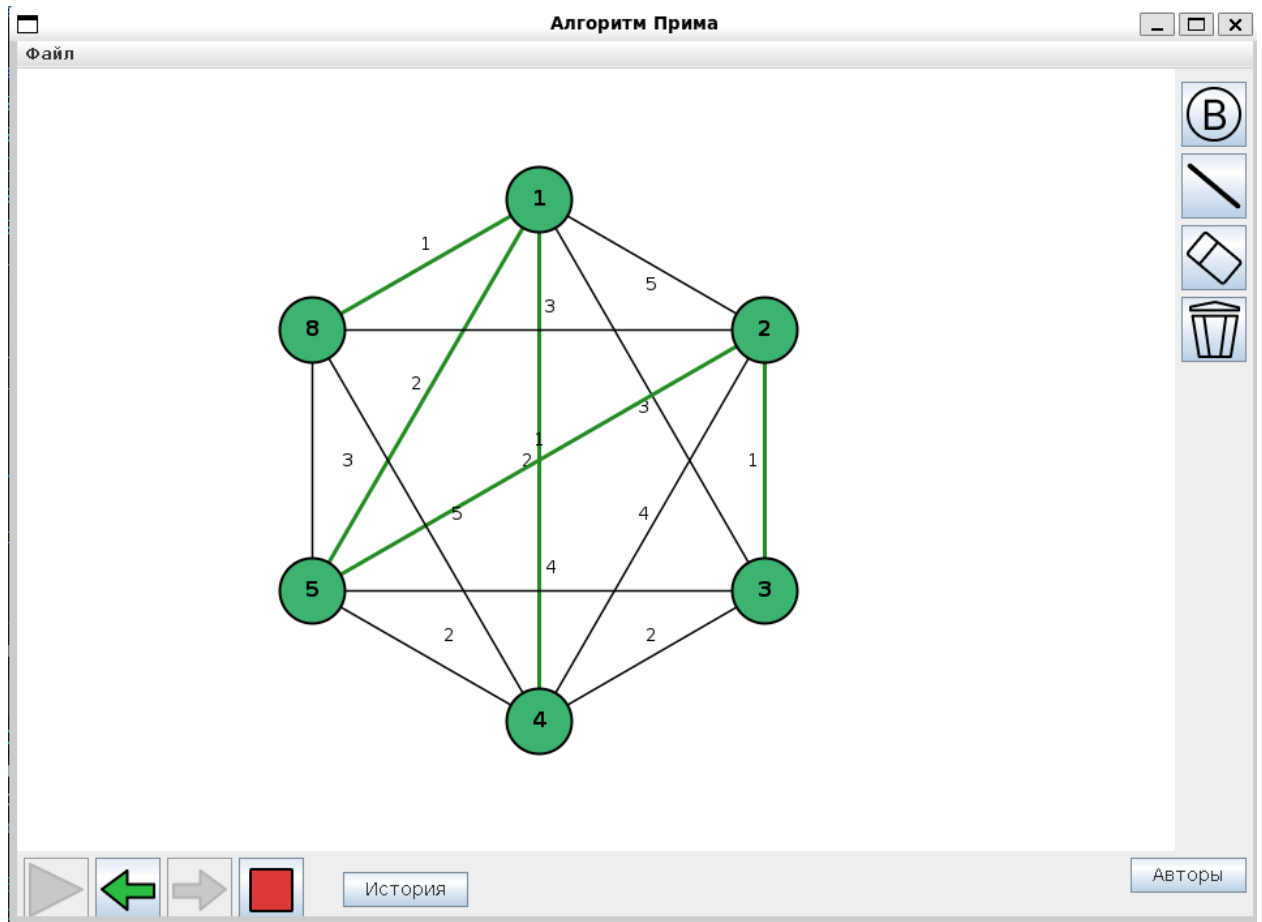


Рисунок 11 - Пользовательский интерфейс